

The background is a blurred image of a computer monitor. It shows a dark-themed code editor with various lines of code in different colors (blue, green, white). At the bottom, a taskbar is visible with several application icons, including what looks like a web browser, a file explorer, and some development tools. The overall image is out of focus, serving as a backdrop for the text.

C#

Level 1

Garbage Collector (GC)



C#

Level 1

Garbage Collector (GC)

🔍 Definition

■ Garbage Collector

- The Garbage Collector (GC) automatically manages managed memory in .NET.
- It identifies objects that are no longer reachable and reclaims the memory they occupy.
- This reduces the need for developers to manually free managed objects.

Explanation

Automatic Memory Management

-  When objects are created, memory is allocated for them.

-  The GC monitors managed objects.

-  When an object is no longer reachable, its memory can eventually be reclaimed.

Create Object



Object Used



Object No Longer Reachable



Garbage Collector



Memory Reclaimed

Main Functions

-  Finds objects that are no longer reachable.

-  Reclaims their managed memory.

-  Helps reduce memory problems caused by forgotten object references.

-  Runs automatically as part of the .NET runtime.

Procedures

1. Create an object.
2. Use the object in the application.
3. Remove the last reachable reference to the object.
4. The object becomes eligible for garbage collection.
5. The GC identifies unreachable objects.
6. The GC reclaims their managed memory.

💡 Example

```
class Student
{
    public string Name;
}
Student student = new Student();
student.Name = "Sara";
student = null;
```

▣ The object that was referenced by `student` may become unreachable after the reference is removed.

▣ The GC can later reclaim its managed memory.

➡ Result:

The developer does not normally need to manually call `free()` for managed objects.

Note

- ◆ GC manages managed memory, not every resource used by an application.
- ◆ Files, database connections, sockets, and other external resources often require explicit disposal.
- ◆ .NET commonly uses `IDisposable` and `using` for deterministic resource cleanup.
- ◆ Garbage collection is automatic, but the exact time when an object is collected is not normally controlled by the programmer.

⊘ Warning

⚡ Do not assume that setting a variable to `null` immediately frees the object's memory.

➡ It only removes that reference. The object becomes eligible for collection if no other reachable references exist.

⚡ Do not confuse garbage collection with complete resource management.

➡ Unmanaged or external resources may require explicit cleanup.

⚡ Do not assume GC prevents every possible memory problem.

➡ Objects that are still reachable can remain in memory even when the application no longer logically needs them.



Comparison

Garbage Collection

- ✓ Automatic management of managed memory
- ✓ Finds unreachable objects
- ✓ Reclaims managed memory
- ✓ Reduces the need for manual memory deallocation
- ✓ Part of the .NET runtime

Manual Resource Disposal

- ✓ Used for resources such as files, streams, sockets, and database connections
- ✓ Often implemented through `IDisposable`
- ✓ Can provide deterministic cleanup
- ✓ Is separate from normal garbage collection

✦ Summary

- ✓ GC = Garbage Collector
- ✓ GC automatically manages managed memory
- ✓ GC identifies unreachable objects
- ✓ GC reclaims memory occupied by eligible managed objects
- ✓ Developers normally do not manually free managed objects
- ✓ Setting a reference to null does not immediately free memory
- ✓ External resources may require explicit disposal
- ✓ `IDisposable` and `using` are commonly used for deterministic resource cleanup
- ✓ Garbage collection is automatic, but collection timing is controlled by the runtime

Thank you 😊